

Conventions to be Followed in a Java Program

In Java, adhering to standard conventions helps improve code readability, maintainability, and collaboration among developers. Following conventions ensures that code is consistent and easily understood by anyone working with it. Below are the important conventions that should be followed when writing Java programs:

1. Naming Conventions

Java has specific rules for naming variables, classes, methods, and other constructs. These rules help distinguish between different types of Java elements and make the code more readable.

a. Class Names

- **Convention:** Class names should be written in **PascalCase** (also known as UpperCamelCase).
- **Example:** `StudentDetails`, `EmployeeManagement`
- Class names should start with a capital letter.
- They should be nouns and should describe the purpose of the class.

b. Method Names

- **Convention:** Method names should be written in **camelCase** (lowerCamelCase).
- **Example:** `calculateSalary()`, `getName()`
- They should start with a lowercase letter and follow the camelCase style for multi-word names.
- Method names should generally be verbs or verb phrases because they represent actions.

c. Variable Names

- **Convention:** Variable names should be written in **camelCase**.
- **Example:** `totalAmount`, `userName`
- Variable names should start with a lowercase letter.
- They should be descriptive and represent the data they hold.

d. Constants

- **Convention:** Constants (final variables) should be written in **UPPERCASE** with underscores separating words.
- **Example:** `PI_VALUE`, `MAX_USER_COUNT`
- Constants should be descriptive and indicate that their value will not change.

e. Package Names

- **Convention:** Package names should be written in **all lowercase**.
- **Example:** `com.companyname.project`, `java.util`
- Package names should be unique and often follow the reverse domain name convention.

f. Interface Names

- **Convention:** Interface names should also follow **PascalCase** (like classes).
 - **Example:** `Readable`, `Serializable`
 - In some cases, they can start with a capital "I" (although this is not commonly followed in modern Java conventions).
-

2. Indentation and Formatting

Proper indentation and formatting are crucial for the readability of the code.

a. Indentation

- **Convention:** Use **4 spaces** for indentation (avoid tabs).
- Each nested block of code should be indented once.
- Example:

```
if (x > 0) {  
    System.out.println("Positive");  
}
```

b. Braces

- **Convention:** The opening brace `{` should be placed at the end of the line (K&R style), and the closing brace `}` should be on its own line.
- **Example:**

```
if (x > 0) {  
    System.out.println("Positive");  
}
```

```
} else {  
    System.out.println("Negative");  
}
```

c. Line Length

- **Convention:** Limit line length to **80-100 characters**.
 - Long lines of code should be broken into multiple lines for better readability.
-

3. Spacing and Blank Lines

a. Spacing

- **Convention:** Leave a space between operators and operands for better readability.
- **Example:**

```
int result = a + b * c; // Correct  
int result=a+b*c;      // Incorrect
```

b. Blank Lines

- **Convention:** Use blank lines to separate logical blocks of code, such as between methods or between different sections of a method.
- **Example:**

```
public class Example {  
    // Method 1  
    public void firstMethod() {  
        // code here  
    }  
  
    // Method 2  
    public void secondMethod() {  
        // code here  
    }  
}
```

4. Avoid Magic Numbers

- **Convention:** Avoid using hardcoded numbers (magic numbers) directly in the code.
- **Example:** Instead of:

```
double area = radius * radius * 3.14159;
```

Use:

```
final double PI = 3.14159;  
double area = radius * radius * PI;
```

5. Access Modifiers

- **Convention:** Always specify access modifiers (e.g., `public`, `private`, `protected`) explicitly.
- **Example:**

```
public class MyClass {  
    private int count; // Declaring variable with an explicit access  
    modifier  
  
    public void increment() {  
        count++;  
    }  
}
```

6. Avoid Using Deprecated Methods/Classes

- **Convention:** Do not use deprecated methods or classes in your program. Always opt for newer, recommended alternatives.
- **Example:** If using `Thread.stop()` is deprecated, use `Thread.interrupt()`.

7. Proper Use of `final` Keyword

- **Convention:** Use the `final` keyword where appropriate, such as for constants, method parameters, or methods that should not be overridden.
- **Example:**

```
public final class FinalClass {  
    // Class cannot be subclassed  
}
```

8. Consistent Use of `this` Keyword

- **Convention:** Use the `this` keyword to refer to instance variables or methods if they are shadowed by method parameters or local variables.
- **Example:**

```
public class Student {  
    private String name;  
  
    public void setName(String name) {  
        this.name = name; // 'this' refers to the instance variable  
    }  
}
```